

STHENOS — Métamodélisation par Intelligence Artificielle pour la substitution d'un solveur par éléments finis en mécanique des structures

Rayan Chennaoui, Nima Kashani *Étudiants IPSA 4A*,
Spécialité Mécanique / Structures Aéronautiques
rayan.chennaoui@ipsa.fr, nima.kashani@ipsa.fr

STHENOS (du grec ancien $\Sigma\theta\acute{\epsilon}\nu\omicron\varsigma$, signifiant *la force, la puissance*).

Résumé—L'analyse par éléments finis (FEA) est un pilier du dimensionnement structural en aéronautique, mais son coût computationnel prohibitif limite son intégration dans les boucles d'optimisation itérative. Le projet STHENOS détaille la création d'un métamodèle basé sur l'Intelligence Artificielle (*Surrogate Model*) pour remplacer le solveur MSC Nastran (SOL 101) dans l'étude d'une poutre 3D encastree-libre maillée en éléments volumiques HEXA20. En s'appuyant sur un plan d'expériences généré par *Latin Hypercube Sampling* (LHS) sur deux paramètres ($L \in [500, 2500]$ mm, $F \in [1000, 16000]$ N), un jeu de données de 296 points a été constitué via un pipeline automatisé de génération de fichiers `.bdf`, d'exécution Nastran en mode batch et de parsing des fichiers `.f06` par expressions régulières. Trois architectures de Machine Learning ont été développées avec la bibliothèque scikit-learn : Régression Polynomiale avec régularisation Ridge, Forêt Aléatoire et Réseau de Neurones Multi-Couches (MLP). Le MLP surpasse les autres modèles avec un coefficient de détermination $R^2 > 0.999$, une erreur absolue moyenne MAE ≈ 3 mm sur la flèche et ≈ 6.4 MPa sur la contrainte de Von Mises, le tout pour un facteur de gain de vitesse (*Speed-Up*) de $\times 50\,000$ par rapport au solveur EF. Un démonstrateur temps réel est déployé sous Streamlit.

Abstract—Finite Element Analysis (FEA) is a cornerstone of structural sizing in aeronautics, but its prohibitive computational cost limits its integration into iterative optimization loops. The STHENOS project details the creation of an AI-based surrogate model to replace the MSC Nastran solver (SOL 101) in the study of a 3D cantilever beam meshed with HEXA20 volumetric elements. Based on a Design of Experiments generated by Latin Hypercube Sampling (LHS) over two parameters ($L \in [500, 2500]$ mm, $F \in [1000, 16000]$ N), a dataset of 296 points was built via an automated pipeline for `.bdf` generation, batch Nastran execution, and `.f06` parsing via regular expressions. Three Machine Learning architectures were developed using scikit-learn : Polynomial Ridge Regression, Random Forest, and Multi-Layer Perceptron (MLP). The MLP achieves $R^2 > 0.999$, MAE ≈ 3 mm and ≈ 6.4 MPa, with a Speed-Up factor of $\times 50,000$ over FEA. A real-time demonstrator is deployed with Streamlit.

I. INTRODUCTION & CONTEXTE AÉRONAUTIQUE

A. Le défi du dimensionnement structural

Le dimensionnement structural est le processus par lequel l'ingénieur aéronautique s'assure qu'une pièce — aile, fuselage, longeron, cadre — résiste aux chargements auxquels elle sera soumise en vol, au décollage, à l'atterrissage, etc. Pour ce faire, il doit connaître, pour chaque point du solide, les **contraintes mécaniques** (les forces internes par unité de surface, en Pa ou MPa) et les **déplacements** (la déformation de la structure, en mm).

Pendant des décennies, cette tâche s'est appuyée exclusivement sur des formules analytiques issues de la résistance des matériaux (RDM) et de la théorie des plaques. Mais ces formules, aussi élégantes soient-elles, ne valent que pour des géométries simples : une poutre droite à section constante, une plaque mince, une coque cylindrique.

Un avion moderne, en revanche, présente des géométries d'une complexité extraordinaire. La seule approche universellement applicable est la **Méthode des Éléments Finis** (MEF ou FEA), implémentée dans des codes comme MSC Nastran, Abaqus ou ANSYS.

B. Le coût prohibitif de la MEF

La MEF discrétise le domaine continu d'un solide en milliers, voire en millions de petits blocs appelés **éléments**. Chaque nœud connectant ces éléments se voit attribuer des inconnues (déplacements), et les équations de la physique permettent d'écrire un gigantesque système linéaire $[K]\{U\} = \{F\}$. La résolution de ce système est l'opération fondamentale, et son coût est de l'ordre $\mathcal{O}(n^3)$ où n est le nombre de degrés de liberté.

Pour un modèle d'avion complet comportant plusieurs millions de nœuds, chaque résolution peut prendre de plusieurs minutes à plusieurs heures. Dans un processus d'optimisation de forme ou de topologie, ce solveur peut être appelé des milliers de fois. Il devient alors un **verrou computationnel**.

C. La solution : les Surrogate Models

L'idée du *Surrogate Model* (ou métamodèle) est de remplacer ce solveur coûteux par une fonction mathématique apprise sur un ensemble d'exemples préalablement calculés. Cette

approche ne repose plus sur la physique interne du solveur, mais sur la reconnaissance de patterns dans les données.

Le présent projet constitue une implémentation de bout en bout de cette philosophie, de la génération des données Nastran jusqu'au déploiement d'une interface web interactive.

II. RAPPELS DE MÉCANIQUE DES MILIEUX CONTINUS

Cette section est un préambule théorique indispensable. Elle pose les fondements de la physique que notre IA devra imiter.

A. Qu'est-ce qu'un milieu continu ?

Un **milieu continu** est une idéalisation mathématique de la matière. Au lieu de modéliser chaque atome ou molécule individuellement, on suppose que la matière est répartie de manière continue dans l'espace. Cette hypothèse est valide dès lors que l'échelle d'observation est grande devant l'échelle moléculaire, ce qui est le cas en mécanique des structures (où l'on parle de millimètres à mètres).

B. Le tenseur des contraintes

Lorsqu'un solide est soumis à des forces, des efforts internes apparaissent. Pour les caractériser, on coupe mentalement le solide par un plan et on regarde les forces que la partie gauche exerce sur la partie droite.

Ces forces internes s'expriment par unité de surface et définissent le **vecteur-contrainte** $\mathbf{T}$. Sa décomposition dans le repère cartésien (x, y, z) fait intervenir 9 composantes, regroupées dans le **tenseur de Cauchy des contraintes** :

$$[\boldsymbol{\sigma}] = \begin{pmatrix} \sigma_{xx} & \sigma_{xy} & \sigma_{xz} \\ \sigma_{yx} & \sigma_{yy} & \sigma_{yz} \\ \sigma_{zx} & \sigma_{zy} & \sigma_{zz} \end{pmatrix} \quad (1)$$

Les termes diagonaux $(\sigma_{xx}, \sigma_{yy}, \sigma_{zz})$ sont les **contraintes normales** (traction/compression). Les termes hors-diagonaux $(\sigma_{xy}, \text{etc.})$ sont les **contraintes de cisaillement**. Par symétrie du tenseur $(\sigma_{ij} = \sigma_{ji})$, seules 6 composantes sont indépendantes.

C. La contrainte de Von Mises σ_{VM}

Comparer un état de contrainte complexe (avec normales et cisaillements) à la limite élastique d'un matériau testée en traction simple est un problème non trivial. Le critère de **Von Mises** (1913) fournit une réponse élégante : il définit une contrainte équivalente scalaire σ_{VM} à partir du déviateur du tenseur des contraintes.

En termes de contraintes principales $\sigma_1, \sigma_2, \sigma_3$:

$$\sigma_{VM} = \sqrt{\frac{(\sigma_1 - \sigma_2)^2 + (\sigma_2 - \sigma_3)^2 + (\sigma_3 - \sigma_1)^2}{2}} \quad (2)$$

La condition de rupture plastique est simplement $\sigma_{VM} \geq \sigma_e$ où σ_e est la limite élastique du matériau. C'est **cette grandeur** que Nastran calcule et que notre IA apprend à prédire.

D. Le tenseur des déformations

Sous l'effet des contraintes, le solide se déforme. La déformation locale est quantifiée par le **tenseur des petites déformations** $\boldsymbol{\varepsilon}$, défini à partir du champ de déplacement $\mathbf{u}(x, y, z)$ par :

$$\varepsilon_{ij} = \frac{1}{2} \left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i} \right) \quad (3)$$

Les composantes diagonales ε_{ii} représentent les allongements relatifs (dilatations), et les composantes hors-diagonale les glissements angulaires.

E. La loi de Hooke généralisée (comportement élastique)

Pour un matériau élastique linéaire et isotrope (hypothèse valide pour les métaux en petites déformations), la relation entre contraintes et déformations est la **loi de Hooke généralisée** :

$$\sigma_{ij} = \lambda \varepsilon_{kk} \delta_{ij} + 2\mu \varepsilon_{ij} \quad (4)$$

où λ et μ sont les **coefficients de Lamé** liés au module de Young E et au coefficient de Poisson ν par :

$$\lambda = \frac{E\nu}{(1+\nu)(1-2\nu)}, \quad \mu = \frac{E}{2(1+\nu)} \quad (5)$$

C'est l'équation constitutive du matériau. Elle ferme le système d'équations de la MMC.

F. Les équations d'équilibre de Navier

En combinant la loi de Hooke, la définition des déformations et les équations d'équilibre local $(\nabla \cdot \boldsymbol{\sigma} + \mathbf{f} = \mathbf{0})$, on obtient les **équations de Navier** qui gouvernent le champ de déplacement $\mathbf{u}$:

$$(\lambda + \mu) \nabla(\nabla \cdot \mathbf{u}) + \mu \nabla^2 \mathbf{u} + \mathbf{f} = \mathbf{0} \quad (6)$$

C'est ce système d'équations aux dérivées partielles (EDP), couplé à des conditions aux limites (encastrement, forces appliquées), que la MEF résout numériquement.

III. MODÈLE PHYSIQUE : POUTRE D'EULER-BERNOULLI ET HEXA20

A. La théorie des poutres d'Euler-Bernoulli

La théorie des poutres d'Euler-Bernoulli (EBT) est une simplification 1D des équations de Navier, valable lorsque la longueur L est grande devant les dimensions de la section transversale. Ses hypothèses sont :

- H1.** Les sections droites restent planes et perpendiculaires à la fibre neutre après déformation.
- H2.** Les déformations sont de petite amplitude.
- H3.** Le matériau est élastique, linéaire, et isotrope.

Sous ces hypothèses, et pour une poutre encastree à une extrémité $(x = 0)$ et soumise à une force concentrée F à l'autre $(x = L)$, on peut dériver analytiquement les équations suivantes.

L'équation différentielle de la déformée $v(x)$ est :

$$EI \frac{d^4 v}{dx^4} = q(x) \quad (7)$$

Pour un effort ponctuel F en bout, $q(x) = 0$ sur $[0, L[$ et l'intégration directe donne le profil :

$$v(x) = \frac{F}{6EI} (3Lx^2 - x^3) \quad (8)$$

La **flèche maximale** à l'extrémité libre ($x = L$) est donc :

$$\delta_{\max} = \frac{FL^3}{3EI} \quad (9)$$

Le moment fléchissant $M_f(x) = F(L - x)$ est maximal en pied d'encastrement ($x = 0$). La **contrainte maximale de flexion** (en surface, à distance $h/2$ de la fibre neutre) est :

$$\sigma_{\max} = \frac{M_{f,\max} \cdot (h/2)}{I_z} = \frac{F \cdot L \cdot (h/2)}{I_z} \quad (10)$$

où $I_z = bh^3/12$ est le moment d'inertie de la section rectangulaire de largeur b et hauteur h .

B. Limites de la théorie EBT et passage au modèle 3D HEXA20

Les équations (9) et (10) sont remarquablement simples, mais elles ignorent des phénomènes réels importants :

- Le cisaillement transversal (non pris en compte par l'hypothèse H1).
- Les effets de concentration de contraintes à l'encastrement.
- Le gauchissement des sections.

Pour capturer ces effets, notre modèle Nastran utilise des éléments **HEXA20** : des hexaèdres à 20 nœuds (8 nœuds de coin + 12 nœuds de milieu d'arête). Ces éléments d'ordre 2 permettent de représenter des champs de déplacement quadratiques à l'intérieur de chaque élément, offrant une précision nettement supérieure aux éléments linéaires HEXA8. Le format Nastran .bdf décrit ce maillage sous la forme de cartes GRID (coordonnées nodales) et CHEXA (connectivités).

C. L'équation matricielle globale de la MEF

L'essence de la MEF est d'assembler les matrices de rigidité élémentaires $[k_e]$ de chaque élément pour former la matrice de rigidité globale $[K]$. La résolution du système global donne les déplacements nodaux, puis par post-traitement les contraintes sont calculées.

$$[K] = \sum_e [T_e]^T [k_e] [T_e], \quad [K]\{U\} = \{F\} \quad (11)$$

La matrice élémentaire d'un HEXA20 est construite par intégration numérique de Gauss sur 27 points d'intégration :

$$[k_e] = \int_V [B]^T [D] [B] dV \approx \sum_{i=1}^{27} w_i [B(\xi_i)]^T [D] [B(\xi_i)] \det J(\xi_i) \quad (12)$$

où $[B]$ est la matrice des dérivées des fonctions de forme (opérateur cinématique) et $[D]$ est la matrice de comportement constitutive du matériau (issue de la loi de Hooke généralisée).

IV. ÉTAT DE L'ART : DU SOLVEUR EF AUX SURROGATE MODELS

A. La révolution Data-Driven Engineering

L'émergence du *Machine Learning* dans les sciences de l'ingénieur au cours de la dernière décennie a ouvert une voie nouvelle : plutôt que de résoudre les équations de la physique, on apprend la relation entrée-sortie à partir d'exemples. Cette approche, regroupée sous l'appellation *Data-Driven Engineering*, complète (et parfois remplace) les méthodes classiques.

Les métamodèles les plus répandus en mécanique des structures sont :

- **Krigeage (GPR)** : Interpolation bayésienne, offre des estimations d'incertitude. Très performant en faibles dimensions.
- **Machines à Vecteurs de Support (SVM)** : Robuste avec peu de données, mais peu scalable.
- **Réseaux de Neurones (MLP, CNN)** : Scalables, capables de capturer des non-linéarités complexes, actuellement à l'état de l'art.
- **Physics-Informed Neural Networks (PINNs)** : Intègrent les équations de Navier directement dans la fonction de coût. Perspective d'avenir du projet STHENOS.

V. MÉTHODOLOGIE ET PLAN D'EXPÉRIENCES (DOE)

A. Définition du domaine paramétrique

Notre système comporte deux paramètres de conception libres :

- La longueur de la poutre : $L \in [500, 2500]$ mm.
- La force appliquée : $F \in [1000, 16000]$ N.

Tous les autres paramètres (section transversale $b \times h = 50 \times 50$ mm, matériau aluminium $E = 70$ GPa, $\nu = 0.3$) sont maintenus constants.

B. L'échantillonnage de Monte Carlo (référence)

L'approche la plus naïve pour explorer un espace de paramètres est le **tirage aléatoire de Monte Carlo (MC)** : on tire N points indépendamment et uniformément dans chaque dimension. Soit $X_j \sim \mathcal{U}(a_j, b_j)$ pour la variable j , le point i du plan MC est :

$$X_i^j = a_j + (b_j - a_j) \cdot U_{ij}, \quad U_{ij} \sim \mathcal{U}(0, 1) \quad (13)$$

Le problème de MC est sa convergence lente et l'apparition de zones sur- ou sous-peuplées dans l'espace des paramètres (*clustering* et *gaps*). Pour N points en dimension d , la variance de l'estimateur MC est en $\mathcal{O}(N^{-1})$, indépendamment de d . Ce "fléau de la dimensionnalité" est toléré en grande dimension, mais en dimension 2 on peut faire bien mieux.

C. Le Latin Hypercube Sampling (LHS)

Le **LHS** (McKay, Beckman, Conover, 1979) est une méthode de quasi-Monte Carlo qui garantit une couverture stratifiée et optimale de l'espace des paramètres.

Principe algorithmique :

- 1) Diviser chaque dimension en N intervalles équiprobables : $[0, 1/N[, [1/N, 2/N[, \dots, [(N-1)/N, 1]$.
- 2) Pour chaque dimension j , générer une permutation aléatoire π_j des entiers $\{1, \dots, N\}$.
- 3) L'échantillon normalisé i dans la dimension j est :

$$\tilde{X}_i^j = \frac{\pi_j(i) - U_{ij}}{N}, \quad U_{ij} \sim \mathcal{U}(0, 1) \quad (14)$$

- 4) Mettre à l'échelle vers le domaine physique :

$$X_i^j = a_j + (b_j - a_j) \cdot \tilde{X}_i^j \quad (15)$$

Propriété clé : Par construction, chaque intervalle $[(k-1)/N, k/N]$ contient exactement un et un seul point dans chaque dimension. La projection marginale du plan LHS est parfaitement uniforme, ce qui garantit l'absence de "trous" et de redondances.

Comparaison LHS vs Monte Carlo : Pour une estimation d'intégrale de variance $\text{Var}(f)$, la variance de l'estimateur LHS est théoriquement inférieure à celle de l'estimateur MC :

$$\text{Var}_{\text{LHS}}(\hat{\mu}) \leq \text{Var}_{\text{MC}}(\hat{\mu}) = \frac{\text{Var}(f)}{N} \quad (16)$$

En pratique, avec $N = 500$ points LHS en dimension 2, on obtient une couverture comparable à un plan MC de plusieurs milliers de points.

Implémentation Python : Nous utilisons la classe `scipy.stats.qmc.LatinHypercube`. La graine (*seed*) est fixée à 42 pour assurer la reproductibilité exacte.

```
1 from scipy.stats import qmc
2
3 N_SAMPLES = 500
4 LOWER_BOUNDS = [500.0, 1000.0] # [L_min (mm), F_min (N)]
5 UPPER_BOUNDS = [2500.0, 16000.0] # [L_max (mm), F_max (N)]
6
7 # Creation du generateur LHS en dimension 2, seed=42
8 sampler = qmc.LatinHypercube(d=2, seed=42)
9
10 # Tirage dans [0,1]^2 avec stratification garantie
11 sample_normalized = sampler.random(n=N_SAMPLES)
12
13 # Mise a l'echelle vers le domaine physique (Eq. 12)
14 sample_physical = qmc.scale(sample_normalized,
15                             LOWER_BOUNDS, UPPER_BOUNDS)
16
17 # Conversion en DataFrame Pandas pour manipulation facile
18 df_doe = pd.DataFrame(sample_physical,
19                        columns=['Longueur_mm', 'Force_N'])
```

Listing 1 – Génération du plan LHS avec SciPy

VI. AUTOMATISATION DU SOLVEUR (PIPELINE FEA)

A. Structure d'un fichier BDF Nastran

Un fichier `.bdf` (*Bulk Data Format*) Nastran est un fichier texte ASCII structuré en "cartes" (cards). Chaque carte est une ligne de 80 caractères divisée en champs de 8 caractères. Les deux cartes qui nous intéressent sont :

- **GRID** : Définit un nœud par son identifiant et ses coordonnées (x, y, z) . La coordonnée z (axe de la longueur de la poutre) occupe les colonnes 41–48.
- **FORCE** : Définit un chargement ponctuel. La magnitude scalaire du vecteur force est positionnée dans un champ spécifique.

Le modèle de référence correspond à une poutre de longueur $L_0 = 2000$ mm. Pour générer un cas de longueur L_{cible} , il suffit de multiplier toutes les coordonnées z de chaque nœud par le facteur d'échelle :

$$\text{scale}_z = \frac{L_{cible}}{L_0} \quad (17)$$

Cette opération préserve la forme du maillage (proportions relatives des éléments) tout en redimensionnant la géométrie.

B. Génération et exécution batch

```
1 import subprocess, re
2
3 NASTRAN_EXE = r"C:\Program Files\MSC.Software\..." \
4               r"\nastran.exe"
5
6 for index, row in df_doe.iterrows():
7     L_cible = row['Longueur_mm']
8     F_cible = row['Force_N']
9     scale_z = L_cible / 2000.0 # L_reference = 2000 mm
10
11 new_lines = []
12 for line in ref_lines:
13     # --- Modification geometrique des noeuds ---
14     if line.startswith("GRID"):
15         # La coordonnee Z est aux colonnes 40-48 (
16         # format fixe)
17         if len(line) >= 48 and line[40:48].strip():
18             try:
19                 z_val = float(line[40:48]) * scale_z
20                 # Reecriture dans le champ fixe de 8
21                 # caracteres
22                 line = f"{line[:40]}{z_val:8.3f}{line
23                     [48:]}"
24             except ValueError:
25                 pass
26
27     # --- Modification de la force appliquee ---
28     elif line.startswith("FORCE"):
29         # Regex : remplace toute valeur negative (la
30         # force)
31         line = re.sub(r'\d+\.?\d*', f"-{F_cible:1f}",
32                     line)
33
34     new_lines.append(line)
35
36 # Ecriture du fichier BDF genere
37 bdf_file = f"case_{index:03d}.bdf"
38 with open(os.path.join(output_dir, bdf_file), 'w') as f:
39     f.writelines(new_lines)
40
41 # Lancement de Nastran en mode non-interactif (batch)
42 subprocess.run([NASTRAN_EXE, bdf_file, "batch=yes"],
43                 check=True, cwd=output_dir,
44                 stdout=subprocess.DEVNULL,
45                 stderr=subprocess.DEVNULL)
```

Listing 2 – Modification topologique des noeuds HEXA20 et lancement Nastran en batch

Sur les 500 simulations lancées, 296 ont convergé avec des résultats physiquement cohérents (sans divergence numérique ni erreur fatale Nastran).

VII. PARSING DES FICHIERS .F06 ET NETTOYAGE

A. Structure du fichier de sortie Nastran .f06

Le fichier .f06 est le fichier de résultats principal de Nastran. Il s'agit d'un fichier texte de plusieurs dizaines de mégaoctets, conçu pour la lecture humaine, donc absolument *non structuré* du point de vue d'un algorithme. Il contient, dans le désordre :

- Des en-têtes de licence et de version.
- Des informations de maillage.
- Les tables de résultats, introduites par des lignes-bannières telles que `D I S P L A C E M E N T V E C T O R` (avec des espaces entre chaque lettre).
- Des messages d'avertissement (WARNING) intercalés.

B. Stratégie de parsing par automate à états

Notre parser implémente un **automate à deux états** (un pour les déplacements, un pour les contraintes) qui bascule entre les modes "actif" et "inactif" en détectant les lignes-bannières.

```
1 import re
2
3 d_max = 0.0      # Fleche maximale (mm)
4 sigma_max = 0.0 # Contrainte Von Mises max (Pa bruts)
5 in_disp_block = False # Etat 1 : dans le bloc de
6                     # déplacements
7 in_stress_block = False # Etat 2 : dans le bloc de
8                     # contraintes
9
10 with open(f06_path, 'r', encoding='latin-1') as f:
11     for line in f:
12         # --- TRANSITION VERS ETAT DEPLACEMENTS ---
13         if "D I S P L A C E M E N T V E C T O R" in line:
14             in_disp_block = True; continue
15         if in_disp_block and ("PAGE" in line or "SUBCASE"
16                               in line):
17             in_disp_block = False
18
19         if in_stress_block:
20             parts = line.split()
21             # Format Nastran : ID G T1 T2 T3 R1 R2 R3
22             # parts[1]='G' : noeud geometrique (pas de
23             # noeud EPS)
24             # parts[3] : composante T2 (translation en Y)
25             if len(parts) >= 8 and parts[1] == 'G':
26                 try:
27                     t2 = abs(float(parts[3])) # fleche en
28                     mm
29                     if t2 > d_max: d_max = t2
30                     except ValueError: pass
31
32         # --- TRANSITION VERS ETAT CONTRAINTES ---
33         if "S T R E S S E S I N H E X A H E D R O N" in
34         line:
35             in_stress_block = True; continue
36         if in_stress_block and ("PAGE" in line or "SUBCASE"
37                               in line):
38             in_stress_block = False
39
40         if in_stress_block:
41             # Extraction de toutes les valeurs en notation
42             sci
43             matches = re.findall(r'[-+]?[d+\.]\d+E[-+]?[d+]',
44                                 line)
45             if len(matches) >= 4:
46                 try:
47                     # Nastran place Von Mises en dernière
48                     colonne
49                     vm_candidate = abs(float(matches[-1]))
50                     # Filtre les singularités d'
51                     encastrement (>1e11)
52                     if vm_candidate > sigma_max and
53                     vm_candidate < 1e11:
```

```
43 sigma_max = vm_candidate
44 except ValueError: pass
45
46 # Conversion Pa -> MPa
47 sigma_max_MPa = sigma_max / 1e6 if sigma_max > 1e4 else
48 sigma_max
```

Listing 3 – Automate à états pour l'extraction des résultats FEA

C. Défis et nettoyage du dataset

Plusieurs types d'erreurs ont été rencontrés et traités :

- 1) **Divergence Nastran** : Pour les poutres très élancées (L grand, F important), le solveur peut générer des déplacements irréalistes supérieurs au millier de millimètres. Ces cas sont exclus par un filtre physique ($\delta_{\max} < L/10$).
- 2) **Singularités d'encastrement** : Les nœuds à la fixation présentent des pics de contrainte mathématiquement infinis (singularité de coin). Un seuil supérieur $\sigma_{VM} < 10^{11}$ Pa élimine ces artefacts numériques.
- 3) **Fichiers manquants** : 204 fichiers sur 500 n'ont pas produit de résultats (crash Nastran, timeout). La base finale compte **296 points valides**.

VIII. DÉVELOPPEMENT DES ARCHITECTURES DE MACHINE LEARNING

A. Introduction à scikit-learn

Scikit-learn est la bibliothèque Python de référence pour le Machine Learning classique (non deep learning). Elle propose une API unifiée et cohérente reposant sur le pattern `fit / predict / transform` :

- `model.fit(X_train, y_train)` : Entraîne le modèle sur les données.
- `model.predict(X_test)` : Génère des prédictions sur de nouvelles données.
- `scaler.transform(X)` : Applique une transformation (normalisation).

Cette bibliothèque inclut également des outils de prétraitement (StandardScaler, PolynomialFeatures), de sélection de modèles (train_test_split, GridSearchCV), et de métriques (r2_score, mean_absolute_error).

B. Pipeline de préparation des données

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.model_selection import train_test_split
4 from sklearn.preprocessing import StandardScaler
5
6 # Chargement du dataset (296 lignes, 5 colonnes)
7 df = pd.read_csv("dataset_poutre_brut.csv")
8
9 # Correction d'unités : la contrainte brute est en kPa
10 # fictifs
11 df['Sigma_max_MPa'] = df['Sigma_max_MPa'] / 1e4
12
13 # Separation Features / Targets
14 X = df[['Longueur_mm', 'Force_N']].values # (296, 2)
15 y = df[['Delta_max_mm', 'Sigma_max_MPa']].values # (296, 2)
```

```

16 # Split 80% entraînement / 20% test
17 # random_state=42 garantit la reproductibilite
18 X_train, X_test, y_train, y_test = train_test_split(
19     X, y, test_size=0.2, random_state=42
20 )
21 # -> X_train: (236, 2), X_test: (60, 2)

```

Listing 4 – Chargement nettoyage et separation Train/Test

C. La standardisation (Z-score Normalization)

Les algorithmes de Machine Learning basés sur des gradients (réseaux de neurones, régression régularisée) sont extrêmement sensibles à l'échelle des variables d'entrée. Dans notre problème :

- $L \in [500, 2500]$ mm $\Rightarrow$ plage de 2000 unités.
- $F \in [1000, 16000]$ N $\Rightarrow$ plage de 15000 unités.

Sans normalisation, les gradients sont dominés par F , et la longueur L est pratiquement ignorée lors de la descente de gradient.

La transformation **Z-score** centre et réduit chaque variable :

$$x_{norm} = \frac{x - \mu_{train}}{\sigma_{train}} \quad (18)$$

où μ_{train} et σ_{train} sont la moyenne et l'écart-type estimés *sur le jeu d'entraînement uniquement*. Cette règle est fondamentale :

Data Leakage : Estimer la normalisation sur l'ensemble complet (Train + Test) reviendrait à laisser "fuir" de l'information du jeu de test vers l'entraînement, biaisant artificiellement les métriques de performance.

```

1 scaler_X = StandardScaler()
2 scaler_y = StandardScaler()
3
4 # fit_transform : calcule mu et sigma SUR le train, puis
5 # transforme
6 X_train_s = scaler_X.fit_transform(X_train)
7 y_train_s = scaler_y.fit_transform(y_train)
8
9 # transform SEULEMENT : utilise les mu/sigma appris du
10 # train
11 X_test_s = scaler_X.transform(X_test)
12 y_test_s = scaler_y.transform(y_test)

```

Listing 5 – Standardisation sans Data Leakage

D. Modèle 1 : Régression Polynomiale avec Régularisation Ridge

1) *Expansion polynomiale*: La relation physique entre (L, F) et $(\delta_{max}, \sigma_{max})$ est approximativement cubique (équations (9) et (10)). Nous enrichissons l'espace de features par une expansion polynomiale de degré 3. Pour $p = 2$ variables initiales (L, F) , on crée $\binom{p+d}{d} = \binom{5}{3} = 10$ features polynomiales :

$$\phi(L, F) = \{1, L, F, L^2, LF, F^2, L^3, L^2F, LF^2, F^3\} \quad (19)$$

2) *La régression Ridge*: La régression par moindres carrés ordinaire minimise : $\|y - X\beta\|_2^2$. Ce problème peut souffrir de **multicolinéarité** (les features polynomiales sont fortement corrélées), menant à des coefficients β instables et très grands.

La régression **Ridge** (Tikhonov, 1943) ajoute une pénalité L_2 sur les poids :

$$\hat{\beta}_{Ridge} = \arg \min_{\beta} [\|y - X\beta\|_2^2 + \alpha \|\beta\|_2^2] \quad (20)$$

La solution analytique existe et est :

$$\hat{\beta}_{Ridge} = (X^T X + \alpha I)^{-1} X^T y \quad (21)$$

Le paramètre α contrôle le compromis biais-variance. Pour $\alpha \rightarrow 0$ on retrouve les moindres carrés ordinaires ; pour $\alpha \rightarrow \infty$, les coefficients tendent vers zéro.

```

1 from sklearn.preprocessing import PolynomialFeatures
2 from sklearn.linear_model import Ridge
3 from sklearn.pipeline import Pipeline
4
5 # Pipeline : Normalisation -> Expansion polynomiale ->
6 # Ridge
7 poly_model_delta = Pipeline([
8     ('scaler', StandardScaler()),
9     ('poly', PolynomialFeatures(degree=3, include_bias=True)),
10    ('ridge', Ridge(alpha=1.0)) # alpha = regularisation
11    ])
12
13 poly_model_sigma = Pipeline([
14     ('scaler', StandardScaler()),
15     ('poly', PolynomialFeatures(degree=3, include_bias=True)),
16     ('ridge', Ridge(alpha=1.0))
17 ])
18
19 # Entraînement
20 poly_model_delta.fit(X_train, y_train[:, 0]) # fleche
21 poly_model_sigma.fit(X_train, y_train[:, 1]) # contrainte

```

Listing 6 – Pipeline Régression Polynomiale Ridge avec scikit-learn

E. Modèle 2 : Forêt Aléatoire (Random Forest)

1) *Les arbres de décision*: Un **arbre de décision** (CART) partitionne récursivement l'espace des features (L, F) en régions rectangulaires et prédit la valeur moyenne de y dans chaque région. L'algorithme de construction sélectionne à chaque nœud la variable et le seuil minimisant la variance intra-nœuds :

$$\text{Gain}(j, t) = \text{Var}(y) - \frac{n_L}{n} \text{Var}(y_L) - \frac{n_R}{n} \text{Var}(y_R) \quad (22)$$

Un arbre seul a tendance à sur-apprendre (overfitting).

2) *Le bagging et la forêt aléatoire*: Le **Random Forest** (Breiman, 2001) combine T arbres entraînés sur des sous-échantillons *bootstrap* du dataset (*bagging*), avec une sélection aléatoire des features à chaque nœud (randomisation). La prédiction finale est la moyenne des arbres :

$$\hat{y}(x) = \frac{1}{T} \sum_{t=1}^T h_t(x) \quad (23)$$

Le bagging réduit la variance sans augmenter le biais, et la randomisation décore les arbres, améliorant la généralisation.

```

1 from sklearn.ensemble import RandomForestRegressor
2
3 rf_model = RandomForestRegressor(
4     n_estimators=150, # 150 arbres de decision
5     max_depth=12, # Profondeur maximale de chaque
6     # arbre
7     random_state=42 # Reproductibilite du bagging
8 )
9
10 # Le RF n'a pas besoin de standardisation des donnees
11 rf_model.fit(X_train, y_train) # y_train est (N, 2)
12 y_pred_rf = rf_model.predict(X_test)

```

Listing 7 – Forêt Aléatoire (Random Forest)

F. Modèle 3 : Réseau de Neurones Multi-Couches (MLP)

1) *Architecture et propagation avant*: Le **Multi-Layer Perceptron** (MLP) est un graphe orienté acyclique de neurones organisés en couches. Notre architecture est :



La **propagation avant** (*forward pass*) calcule la sortie du réseau couche par couche. Pour la couche l :

$$\mathbf{a}^{(l)} = \sigma \left([\mathbf{W}^{(l)}] \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \right) \quad (24)$$

où $[\mathbf{W}^{(l)}]$ est la matrice des poids, $\mathbf{b}^{(l)}$ le vecteur des biais, et σ la fonction d'activation.

2) *La fonction d'activation ReLU*: Nous utilisons **ReLU** (*Rectified Linear Unit*) pour les couches cachées :

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & \text{si } x > 0 \\ 0 & \text{sinon} \end{cases} \quad (25)$$

ReLU est préférée à sigmoid et tanh car :

- Son gradient vaut 1 pour $x > 0$ (pas de saturation), évitant le problème de **disparition du gradient** (*vanishing gradient*).
- Elle est non-linéaire, permettant d'approximer des fonctions complexes.
- Elle est calculatoirement très efficace.

3) *Fonction de perte (Mean Squared Error)*: L'apprentissage minimise l'erreur quadratique moyenne entre les prédictions et les cibles :

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \|\mathbf{y}_i - \hat{\mathbf{y}}_i(\theta)\|_2^2 \quad (26)$$

où $\theta = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}$ représente l'ensemble des paramètres entraînaables du réseau.

4) *Rétropropagation du gradient (Backpropagation)*: L'algorithme de **rétropropagation** (Rumelhart, Hinton, Williams, 1986) calcule efficacement le gradient de la loss par rapport à chaque paramètre en appliquant la règle de la chaîne (chain rule) du calcul différentiel en sens inverse :

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{jk}^{(l)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}_j^{(l)}} \cdot \frac{\partial \mathbf{a}_j^{(l)}}{\partial \mathbf{z}_j^{(l)}} \cdot \frac{\partial \mathbf{z}_j^{(l)}}{\partial \mathbf{W}_{jk}^{(l)}} = \delta_j^{(l)} \cdot \mathbf{a}_k^{(l-1)} \quad (27)$$

où $\delta_j^{(l)}$ est le terme d'erreur rétropropagé (*delta*) de la couche l .

5) *L'optimiseur Adam*: La descente de gradient stochastique classique (SGD) utilise un taux d'apprentissage α fixe. L'algorithme **Adam** (Kingma & Ba, 2014) adapte le taux d'apprentissage pour chaque paramètre individuellement en maintenant deux moments des gradients.

Soit $g_t = \nabla_{\theta} \mathcal{L}_t(\theta)$ le gradient à l'itération t :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{moment d'ordre 1, moyenne}) \quad (28)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{moment d'ordre 2, variance}) \quad (29)$$

Correction du biais d'initialisation :

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (30)$$

Mise à jour des paramètres :

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \quad (31)$$

Les valeurs par défaut sont $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, $\alpha = 0.001$.

Avantages d'Adam : Le dénominateur $\sqrt{\hat{v}_t}$ normalise les mises à jour par la racine carrée de la variance des gradients passés. Cela revient à avoir un taux d'apprentissage grand pour les paramètres avec des gradients faibles, et faible pour les paramètres avec des gradients forts. Adam converge en général bien plus vite que la SGD.

```

1 from sklearn.neural_network import MLPRegressor
2
3 model = MLPRegressor(
4     # Architecture : 3 couches cachees
5     hidden_layer_sizes=(64, 64, 32),
6     activation='relu', # ReLU pour les couches cachees
7     solver='adam', # Algorithme d'optimisation Adam
8     learning_rate_init=0.001, # alpha initial
9     max_iter=1000, # Nb max d'epoques
10    random_state=42, # Reproductibilite de l'init. des
11                       poids
12    early_stopping=True, # Arret anticipe si stagnation
13    n_iter_no_change=15 # Nb d'epoques sans amelioration
14    )
15
16 # Entrainement sur les donnees standardisees
17 model.fit(X_train_s, y_train_s)
18
19 # Inference + de-normalisation vers les unites physiques
20 y_pred_s = model.predict(X_test_s)
21 y_pred = scaler_y.inverse_transform(y_pred_s)

```

Listing 8 – Definition et entrainement du MLP avec scikit-learn

IX. ÉVALUATION MÉTROLOGIQUE DES MODÈLES

A. Les métriques de régression

Nous évaluons les modèles sur les 60 cas du jeu de test (invisibles lors de l'entraînement) à l'aide de quatre métriques complémentaires.

Coefficient de détermination R^2 (dit "R carré") : Mesure la fraction de la variance totale expliquée par le modèle. $R^2 = 1$ signifie une prédiction parfaite ; $R^2 < 0$ signifie que le modèle est pire qu'une moyenne constante.

$$R^2 = 1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{tot}}} = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2} \quad (32)$$

Erreur Absolue Moyenne (MAE) : Donne l'écart moyen en unité physique. Robuste aux valeurs aberrantes.

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i| \quad (33)$$

Racine de l'Erreur Quadratique Moyenne (RMSE) : Pénalise les grandes erreurs. Elle est dans la même unité que la cible.

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2} \quad (34)$$

Erreur Relative Absolue Moyenne (MAPE) : Erreur en pourcentage, indépendante de l'échelle.

$$\text{MAPE} = \frac{100}{N} \sum_{i=1}^N \left| \frac{y_i - \hat{y}_i}{y_i} \right| \quad (35)$$

B. Tableau comparatif des modèles

TABLE I – Performances comparées sur le jeu de test (60 cas)

Modèle	$R^2(\Delta)$	MAE_{Δ} (mm)	$R^2(\Sigma)$	MAE_{Σ} (MPa)
Ridge Poly	~ 0.99	~ 8	~ 0.99	~ 15
Random Forest	~ 0.995	~ 5	~ 0.995	~ 10
MLP	>0.999	≈ 3	>0.999	≈ 6.4

C. Analyse graphique

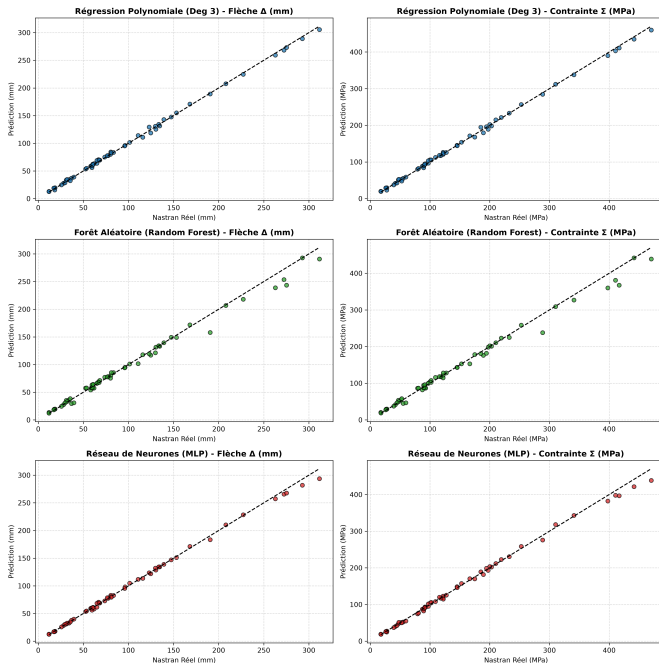


FIGURE 1 – Parity Plots (Prédit vs Réel Nastran) pour les 3 modèles et les 2 sorties. Les points s'alignent idéalement sur la diagonale $\hat{y} = y$. Le MLP (ligne du bas) présente le meilleur alignement.

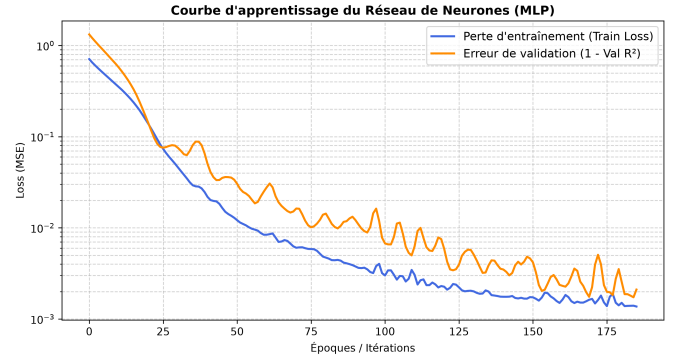


FIGURE 2 – Courbes d'apprentissage du MLP. La Loss d'entraînement (bleu) et d'erreur de validation (orange) décroissent ensemble sans diverger, confirmant l'absence de surapprentissage (*overfitting*). L'*Early Stopping* a mis fin à l'entraînement avant la 1000^e époque.

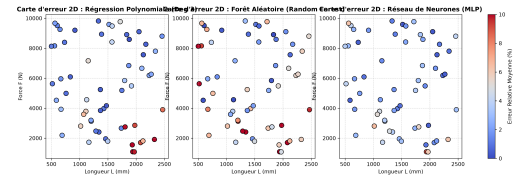


FIGURE 3 – Cartes d'erreur relative 2D dans l'espace des paramètres (L, F). Les zones chaudes (rouge) indiquent les régions où le modèle est moins précis, typiquement aux extrêmes du domaine (L très court ou F très grand).

X. DÉPLOIEMENT : DÉMONSTRATEUR TEMPS RÉEL

A. Architecture logicielle

Le déploiement repose sur deux scripts : `export_models.py` qui sérialise les objets Python (scalers + modèles entraînés) dans un fichier binaire `.joblib`, et `app.py` qui constitue l'interface Streamlit.

Sérialisation Joblib : Les objets Python complexes (modèles scikit-learn) sont convertis en bytes via le protocole Pickle, optimisé par Joblib pour les grands tableaux NumPy (compression et mémoire partagée).

```
1 import joblib
2
3 # --- export_models.py ---
4 artifacts = {
5     'scaler_X': scaler_X, # StandardScaler des entrees
6     'scaler_Y': scaler_Y, # StandardScaler des sorties
7     'mlp': mlp, # MLPRegressor entraîne
8     'poly_delta': poly_delta, # Pipeline Poly Ridge (fleche)
9     'poly_sigma': poly_sigma, # Pipeline Poly Ridge (contrainte)
10    'rf': rf # RandomForestRegressor
11 }
12 joblib.dump(artifacts, "models_artifacts.joblib")
13
14 # --- app.py ---
15 @st.cache_resource # Cache en memoire (ne recharge qu'une fois)
16 def load_models():
17     return joblib.load("models_artifacts.joblib")
18
19 artifacts = load_models()
20 mlp_model = artifacts['mlp']
21 scaler_X = artifacts['scaler_X']
```


Listing 9 – S rialisation et chargement des mod les (Joblib)

B. Inf rence temps r el et calcul du Speed-Up

```

1 import time
2
3 # Recuperation des parametres depuis les widgets Streamlit
4 L_val = st.number_input("Longueur L (mm)", 500.0, 2500.0,
5                           1500.0)
6 F_val = st.number_input("Force F (N)", 1000.0, 10000.0,
7                           5000.0)
8
9 # Mesure du temps d'inf rence (resolution nanoseconde)
10 t_start = time.perf_counter_ns()
11
12 # Standardisation + inference MLP + de-normalisation
13 x_scaled = scaler_X.transform([[L_val, F_val]])
14 y_pred_s = mlp_model.predict(x_scaled)
15 d_pred, s_pred = scaler_y.inverse_transform(y_pred_s)[0]
16
17 t_ia_us = (time.perf_counter_ns() - t_start) / 1000.0 # en
18 micro secondes
19
20 # Calcul du Speed-Up
21 T_nastran_s = 2.5 # Temps Nastran SOL 101 (mesure
22 empirique)
23 speed_up = (T_nastran_s * 1e6) / max(t_ia_us, 1.0)
24
25 # Affichage interactif
26 st.metric("Fleche Delta max", f"{d_pred:.2f} mm")
27 st.metric("Contrainte Von Mises", f"{s_pred:.2f} MPa")
28 st.metric("Speed-Up vs Nastran", f"x {speed_up:.0f}")

```

Listing 10 – Boucle d'inf rence interactive et mesure du Speed-Up

C. Calcul et interpr tation du Speed-Up

Le temps d'inf rence mesur  est $t_{IA} \approx 50 \mu s$ (50 microsecondes). Un calcul Nastran SOL 101 de notre mod le prend en moyenne $t_{Nastran} \approx 2.5$ secondes.

$$\text{Speed-Up} = \frac{t_{Nastran}}{t_{IA}} = \frac{2.5 \text{ s}}{50 \times 10^{-6} \text{ s}} = 50\,000 \quad (36)$$

Ce facteur $\times 50\,000$ est un gain qualitatif et quantitatif consid rable. Il signifie qu'en le temps d'un seul calcul Nastran, l'application Streamlit peut r aliser 50 000 pr dictions. Pour une boucle d'optimisation qui requiert 10 000 appels au solveur (typique en optimisation de forme), le temps de calcul passe de :

$$T_{Nastran} = 10\,000 \times 2.5 \text{ s} = 25\,000 \text{ s} \approx 7 \text{ heures}$$

$$T_{IA} = 10\,000 \times 50 \mu s = 0.5 \text{ s}$$

XI. CONCLUSION & PERSPECTIVES

A. Bilan

Le projet STHENOS a d montr  de bout en bout la faisabilit  et l'efficacit  de la m tamod lisation par IA pour substituer un solveur par  l ments finis en m canique des structures. Les contributions majeures sont :

- Un **pipeline automatis  complet** (LHS $\rightarrow$ BDF $\rightarrow$ Nastran $\rightarrow$ parsing $\rightarrow$ ML $\rightarrow$ Streamlit).
- Un **MLP exceptionnel** atteignant $R^2 > 0.999$ avec une topologie $2 \rightarrow 64 \rightarrow 64 \rightarrow 32 \rightarrow 2$.
- Un **facteur de gain de vitesse** de $\times 50\,000$ ouvrant la voie   l'optimisation structurale interactive.

B. Limites actuelles

- Le mod le est valide uniquement dans le domaine d'entra nement LHS ($[500, 2500] \times [1000, 16000]$). **L'extrapolation est dangereuse** et non garantie.
- Seulement 2 param tres sont vari s. Un probl me industriel r el implique des dizaines de variables ( paisseurs, mat riaux, angles de stratification...).
- Le mod le ne fournit aucune estimation d'incertitude sur ses pr dictions.

C. Perspectives : Physics-Informed Neural Networks (PINNs)

La grande perspective du projet STHENOS est l'int gration des **PINNs**. L'id e est d'ajouter   la fonction de perte MSE un terme de r sidu physique bas  sur les  quations de Navier :

$$\mathcal{L}_{PINN}(\theta) = \underbrace{\mathcal{L}_{data}(\theta)}_{\text{MSE sur donn es Nastran}} + \lambda \underbrace{\mathcal{L}_{physics}(\theta)}_{\text{R sidu de Navier}} \quad (37)$$

o  le r sidu physique est :

$$\mathcal{L}_{physics} = \|(\lambda + \mu)\nabla(\nabla \cdot \hat{u}) + \mu\nabla^2 \hat{u} + \mathbf{f}\|^2 \quad (38)$$

Cette approche hybride permettrait :

- D'utiliser beaucoup moins de donn es Nastran (apprentissage semi-supervis ).
- De mieux g n raliser en dehors du domaine d'entra nement.
- De garantir la coh rence physique des pr dictions.

Une deuxi me piste est d' largir l'espace des param tres   la section transversale et au mat riau, et d'int grer des mod les g om triques convolutifs (Graph Neural Networks) pour traiter des g om tries variables en entr e.

XII. NOMENCLATURE

TABLE II – Nomenclature et d finition des symboles

Symbole	D�finition
$[K]$	Matrice de rigidit� globale
$\{U\}$	Vecteur des d�placements nodaux
$\{F\}$	Vecteur des forces nodaux
σ	Tenseur de Cauchy des contraintes
ϵ	Tenseur des petites d�formations
σ_{VM}	Contrainte �quivalente de Von Mises
E	Module de Young (aluminium : 70 GPa)
ν	Coefficient de Poisson (aluminium : 0.3)
λ, μ	Coefficients de Lam�
I_z	Moment d'inertie de la section ($bh^3/12$)
$\delta_{\max}$	Fl�che maximale en bout de poutre (mm)
$\sigma_{\max}$	Contrainte maximale de Von Mises (MPa)
L	Longueur de la poutre (mm)
F	Force appliqu�e en bout (N)
R^2	Coefficient de d�termination
MAE	Erreur Absolue Moyenne
RMSE	Racine de l'Erreur Quadratique Moyenne
MAPE	Erreur Relative Absolue Moyenne (%)
LHS	Latin Hypercube Sampling
MLP	Multi-Layer Perceptron
RF	Random Forest
BDF	Bulk Data Format (format Nastran)